

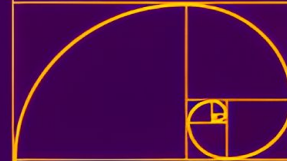
Hashing Introduction

CSCI 33500: Software Design and Analysis III

Sadab Hafiz

sh3646@hunter.cuny.edu

10



Motivation

Unordered Data

- In many applications, data ordering is not relevant and what matters is quick access to the data
- In such instances, we want to be able to access the data in $\Theta(1)$ time
- Recall, what you already know:
 - ◆ Sequence Containers:
 - Typically $\Theta(n)$ to find an item
 - ◆ Associative Containers:
 - Ordered using balanced BST
 - Typically $\Theta(\log(n))$ to find an item
- Can we do better?

Example

Player Information



- Let's say you are an assistant manager at a football club 🏑
- You got hired recently and don't know information about all the players on top of your head
- The main manager is **very strict** and will **start yelling at you if you take too long** to respond to him
- Your goal is to design a system which will allow you to look up player information in $\Theta(1)$:
 - ◆ **Jersey number** is the key (unique)
 - ◆ E.g., player[10] = {"Name": "Messi", ...}
 - ◆ E.g., player[07] = {"Name": "Ronaldo", ...}
 - ◆ E.g., player[11] = {"Injury": "6 months", ...}

Example

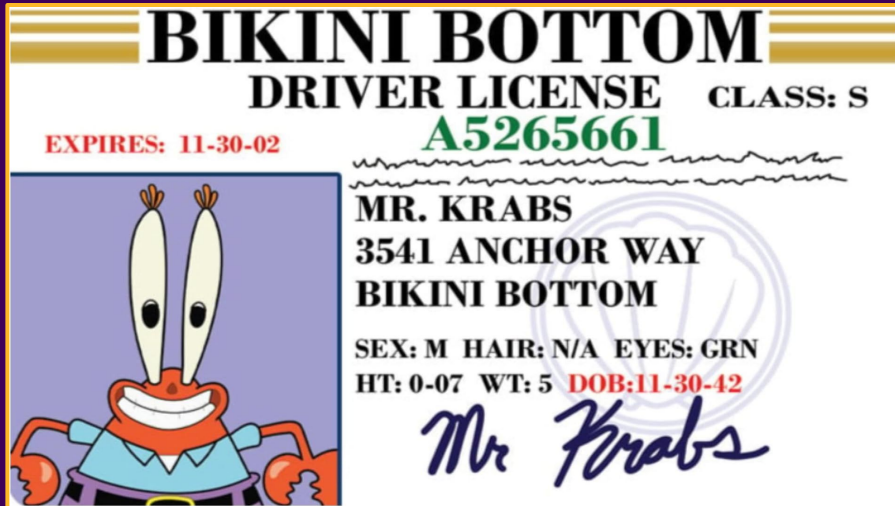
Student Records



- Jersey numbers are:
 - ◆ Unique to each player (0 - 99)
 - ◆ Typically teams have 25 players
 - ◆ We can simply use an array of size 99
- Let's say we are designing a fast lookup system for student records:
 - ◆ We want $\Theta(1)$ access
 - ◆ Student ID as key:
 - Unique identifier
 - Hunter ID is an 8-digit number
 - ◆ E.g., Student[10242000] =
 - {"Name": "Chester Bennington"}
 - {"Major": "Music"}
 - {"Year": "Sophomore"}
 - {"Band": "Linkin Park"}

Example

Storage Considerations



- Can we store this in an array?
- 10^8 possible student ID:
 - ◆ Let's say each ID takes 8 bytes
 - ◆ Would require $10^8 \times 8$ bytes:
 - 800,000,000 bytes
 - About 800 MB
 - ◆ Very inefficient use of space
 - 22538 students enrolled (in 2026)
 - Need to store 22538 ID
 - $22538 \times 8 = 180304$ bytes
 - Less than 0.2 MB necessary
 - ◆ $(800 - 0.2) / 800 = 99.98\%$ empty

Example

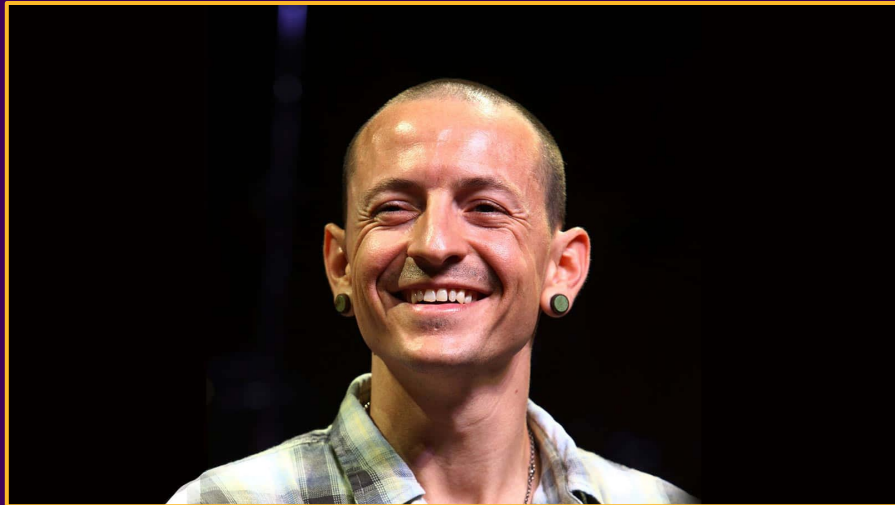
Tracking ID



- 800 MB nowadays is not a lot
- What about 22-character keys?
 - ◆ E.g., [USPS tracking number](#)
 - A - Z and 0 - 9
 - 36 possible value for each position
 - E.g., [K7R29B8Q1M4P5X6V0N9L3Z](#)
 - Data can be shipment information
 - Arrival time
 - Package weight
 - Tracking history
 - ◆ Let's say each key requires 8 bytes
 - $36^{22} \times 8 = \text{IMPOSSIBLE}$
- Goal: $\Theta(1)$ access with $\Theta(n)$ space

Example

Student Records



- Instead of a massive array of 10^8 size, we can create a 1000 size array:
 - ◆ How can we convert an 8-digit number into a 3-digit number?
 - ◆ First three digits might cause a problem:
 - Many student IDs start with:
 - 201, 202, 203, ...
 - ◆ The last three digits could work
 - Essentially random
- Therefore, we can access Chester's information using `Student[000]`

Example

Student Records

⋮	⋮
000	Chester
001	
002	
003	
004	79
005	
⋮	⋮

→ What is the likelihood that in a class of size 100, no two students will share the same last three digits?

→ **Not very high:**

$$1 \cdot \frac{999}{1000} \cdot \frac{998}{1000} \cdot \frac{997}{1000} \cdot \dots \cdot \frac{901}{1000} \approx 0.005959$$

→ Consequently, we may write a function that maps a student ID onto a 3-digit number

- ◆ We can store records in that location
- ◆ $\Theta(1)$ operations:
 - Insert, Access, Erase

→ Problem: two or more student IDs may map to the same number

- ◆ Chester Bennington ID: [10242000](#)
- ◆ Mike Shinoda ID: [23252000](#)

Hashing

Definition

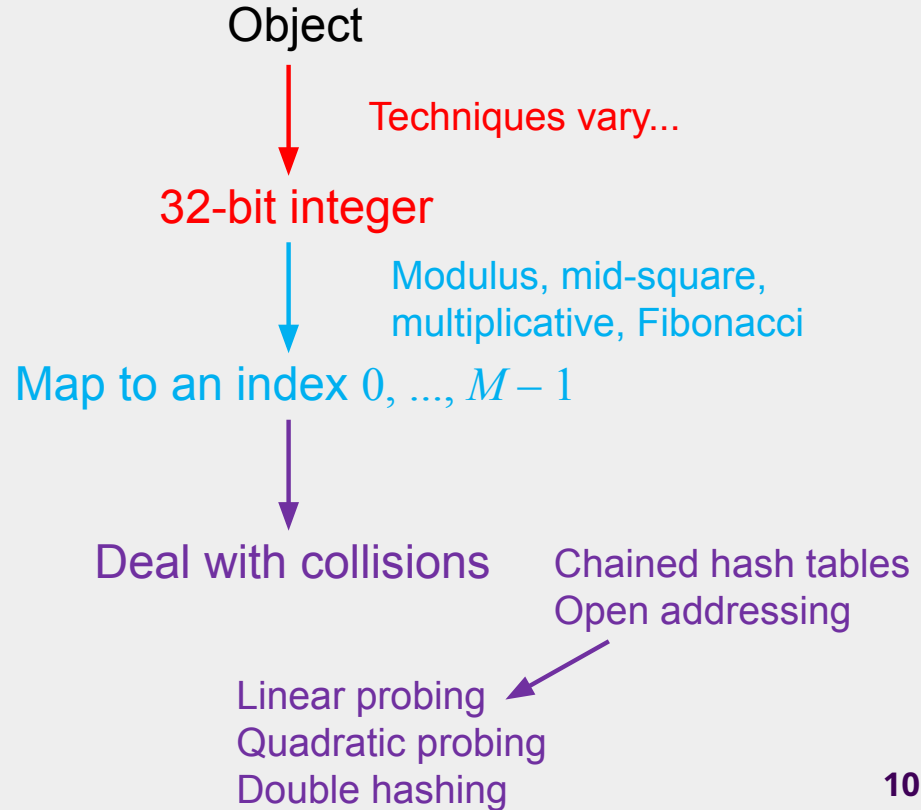
- The process of mapping an object or a number onto an integer in a given range is called *hashing*
- Problem: multiple objects may hash to the same value
 - ◆ Such an event is called a *collision*
- Hash tables use a **hash function** together with a mechanism for **dealing with collisions**
 - ◆ As shown in student record example, **collisions will most definitely occur** when storing a large amount of data

Hashing

Process

→ We will break the process into three independent steps:

- ◆ We will try to get them down to $\Theta(1)$

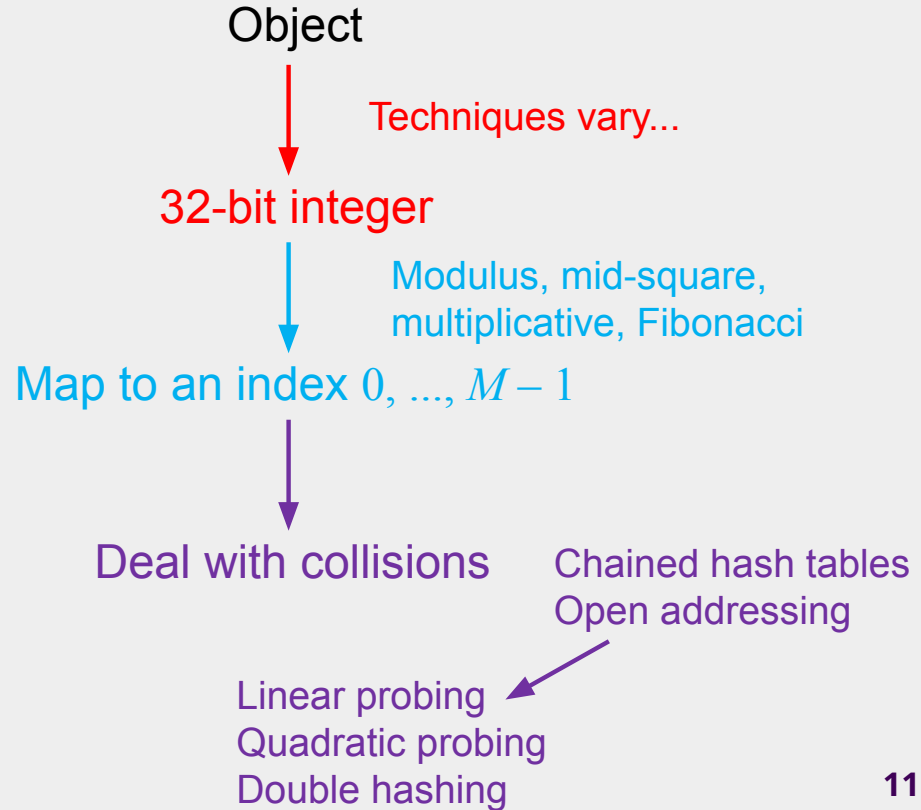


Hashing

Process

→ The ultimate goal is to map the object onto an integer range:

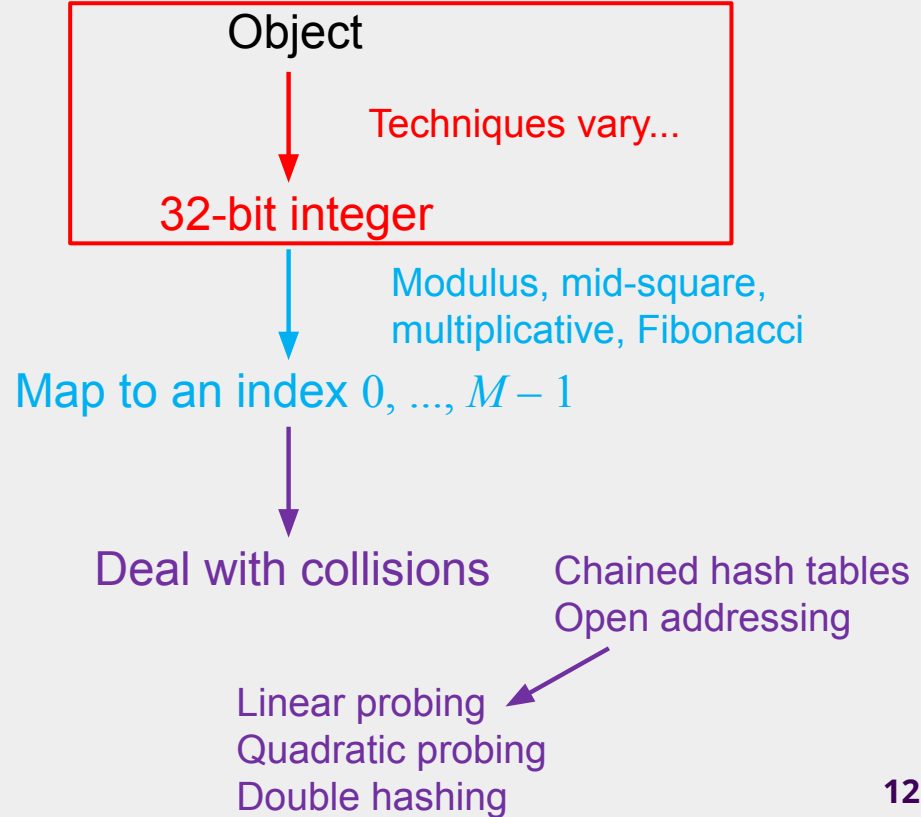
$0, 1, 2, \dots, M - 1$



Hashing

Process

- Let's look at the first problem
 - ◆ Hashing an object into a 32-bit integer



Hash Function

Properties

- Necessary properties of a hash function h are:
 - ◆ Should be fast: ideally $\Theta(1)$
 - ◆ The hash value must be *deterministic*
 - It must always return the same 32-bit integer each time
 - ◆ Equal objects hash to equal values
 - $x = y \Rightarrow h(x) = h(y)$
 - ◆ If two objects are randomly chosen, there should be only a one-in- 2^{32} chance that they have the same hash value
- We will look at two classes of h :
 - ◆ Predetermined hash functions (explicit)
 - ◆ Arithmetic hash functions (implicit)

Predetermined Hash Functions

→ The easiest solution is to give each object a unique number

```
class Class_name {  
    private:  
        unsigned int hash_value; // int:           $-2^{31}, \dots, 2^{31} - 1$   
                                // unsigned int:   $0, \dots, 2^{32} - 1$   
  
    public:  
        Class_name();  
        unsigned int hash() const;  
};  
  
Class_name::Class_name() {  
    hash_value = ???;  
}  
  
unsigned int Class_name::hash() const {  
    return hash_value;  
}
```

Predetermined Hash Functions

→ For example, an auto-incremented static member variable

```
class Class_name {  
    private:  
        unsigned int hash_value;  
        static unsigned int hash_count;  
    public:  
        Class_name();  
        unsigned int hash() const;  
};
```

```
unsigned int Class_name::hash_count = 0;
```

```
Class_name::Class_name() {  
    hash_value = hash_count;  
    ++hash_count;  
}
```

```
unsigned int Class_name::hash() const {  
    return hash_value;  
}
```

Predetermined Hash Function Examples

- Almost all Hunter students have two primary identifiers which can serve as keys for a hash function:
 - ◆ Hunter student ID (8-digit decimal number)
 - ◆ Social Security Number (9-digit decimal number)
- How many bits are required to store a 9-digit number?
 - ◆ Number of possible 9-digit numbers: 10^9
 - ◆ Number of bits required to represent (convert to binary):
 - $\log_2(10^9) = 29.897$
 - **Less than 32-bits**
- Therefore, a 32-bit integer will be enough to hash the above keys
- The key is used by the hash function to generate a hash value

Predetermined Hash Functions

- If we only need the hash value while the object exists in memory, we can use the address of the object:

```
unsigned int Class_name::hash() const {  
    return reinterpret_cast<unsigned int>( this );  
}
```

- This fails if an object may be stored in secondary memory
 - ◆ It will have a different address the next time it is loaded

Predetermined Hash Functions

- Predetermined hash values give each object a unique hash value
- This is **not always appropriate**:
 - ◆ Objects which are conceptually equal
 - ◆ E.g., Class used to represent rational numbers
 - `Rational x(1, 2);`
 - `Rational y(3, 6);`
 - ◆ Strings with the same characters:
 - `string str1 = "Hello world!";`
 - `string str2 = "Hello world!";`
- We expect conceptually equal objects to have the same hash value
- These hash values must depend on the data members
 - ◆ Usually this uses arithmetic functions

Arithmetic Hash Function

Using Data Members

- An arithmetic hash function is:
 - ◆ A deterministic function
 - ◆ Calculates the hash values from relevant data members of an object
- Next, arithmetic functions for:
 - ◆ Rational numbers
 - ◆ Strings

Rational Number Class

→ What if we just add the numerator and denominator?

```
class Rational {  
    private:  
        int numer, denom;  
    public:  
        Rational( int, int );  
};  
  
unsigned int Rational::hash() const {  
    return static_cast<unsigned int>( numer ) +  
           static_cast<unsigned int>( denom );  
}
```

Rational Number Class

→ Addition is commutative resulting in **non-unique hash values** (x/y vs y/x)

```
class Rational {  
    private:  
        int numer, denom;  
    public:  
        Rational( int, int );  
};  
  
unsigned int Rational::hash() const {  
    return static_cast<unsigned int>( numer ) +  
           static_cast<unsigned int>( denom );  
}
```

Rational Number Class

→ We could improve on this: multiply the denominator by a large prime

```
class Rational {  
    private:  
        int numer, denom;  
    public:  
        Rational( int, int );  
};  
  
unsigned int Rational::hash() const {  
    return static_cast<unsigned int>( numer ) +  
           429496751*static_cast<unsigned int>( denom );  
}
```

Rational Number Class

For example, the output of:

```
int main() {  
    cout << Rational( 0, 1 ).hash() << endl;  
    cout << Rational( 1, 2 ).hash() << endl;  
    cout << Rational( 2, 3 ).hash() << endl;  
    cout << Rational( 99, 100 ).hash() << endl;  
}
```

is

```
429496751  
858993503  
1288490255  
2239
```

→ Recall that arithmetic operations wrap on overflow (back to 0 after max)

Rational Number Class

→ This hash function **does not generate unique values**

→ The following pairs have the same hash values:

0/1 1327433019/800977868

1/2 534326814/1480277007

2/3 820039962/1486995867

→ Finding rational numbers with matching hash values is very difficult:

◆ Finding these required the generation of 1,500,000,000 random rational numbers

→ It is fast: $\Theta(1)$

→ It does produce an even distribution

Rational Number Class

Problem: the rational numbers 1/2 and 2/4 have different hash values

The output of:

```
int main() {  
    cout << Rational( 1, 2 ).hash();  
    cout << Rational( 2, 4 ).hash();  
}
```

is

```
858993503  
1717987006
```

Rational Number Class

→ Solution: divide through by the Greatest Common Divisor (GCD)

```
Rational::Rational( int a, int b ):numer(a), denom(b) {  
    int divisor = gcd( numer, denom );  
    numer /= divisor;  
    denom /= divisor;  
}
```

→ Euclidean algorithm to find GCD

◆ $\Theta(\log(\min(|a|, |b|)))$

```
int gcd( int a, int b) {  
    while( true ) {  
        if ( a == 0 ) {  
            return (b >= 0) ? b : -b;  
        }  
  
        b %= a;  
  
        if ( b == 0 ) {  
            return (a >= 0) ? a : -a;  
        }  
  
        a %= b;  
    }  
}
```

Rational Number Class

Problem: the rational numbers $1/2$ and $-1/-2$ have different hash values

The output of:

```
int main() {  
    cout << Rational( 1, 2 ).hash();  
    cout << Rational( -1, -2 ).hash();  
}
```

is

```
858993503  
3435973793
```

Rational Number Class

→ Solution: define a normal form

- ◆ Require that the denominator is positive

```
Rational::Rational( int a, int b ):numer(a), denom(b) {  
    int divisor = gcd( numer, denom );  
    divisor = (denom >= 0) ? divisor : -divisor;  
    numer /= divisor;  
    denom /= divisor;  
}
```

String Class

Arithmetic Hash Function

- Two strings are equal if:
 - ◆ All the characters are equal
 - ◆ All the characters are in identical order
- A string is simply an array of bytes:
 - ◆ Each byte stores a value from 0 to 255
 - ◆ An encoding is a set of rules that tells the computer how to interpret these values
 - E.g., ASCII, UTF-8, etc.
- Any hash function must be a function of these bytes

String Class

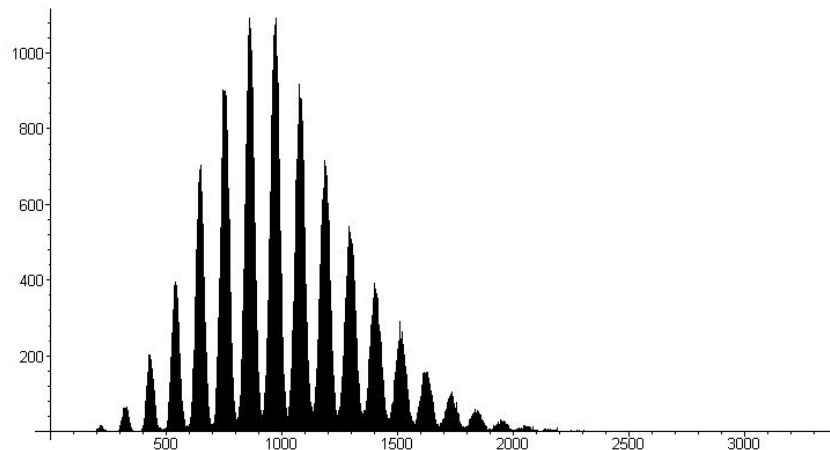
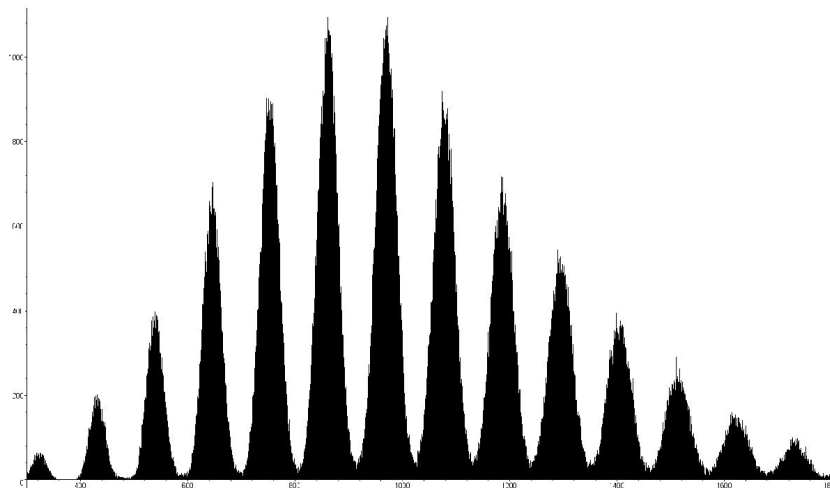
→ We could, for example, just add the characters:

```
unsigned int hash( const string &str ) {  
    unsigned int hash_value = 0;  
  
    for ( int k = 0; k < str.length(); ++k ) {  
        hash_value += str[k];  
    }  
  
    return hash_value;  
}
```

String Class

→ This approach is not very good:

- ◆ Slow run time: $\Theta(n)$
- ◆ Words with the same characters hash to the same code:
 - “form” and “from”
- ◆ A poor distribution
 - E.g., all words in MobyTM Words II by Grady Ward ([single.txt](#)) [Project Gutenberg](#)
 - Bias towards some ranges
 - A lot of collisions



String Class

→ Let individual characters represent the coefficients of a polynomial in x :

$$p(x) = c_0 x^{n-1} + c_1 x^{n-2} + \cdots + c_{n-3} x^2 + c_{n-2} x + c_{n-1}$$

→ Use [Horner's rule](#) to evaluate this polynomial at a prime number

◆ E.g., $x = 12347$

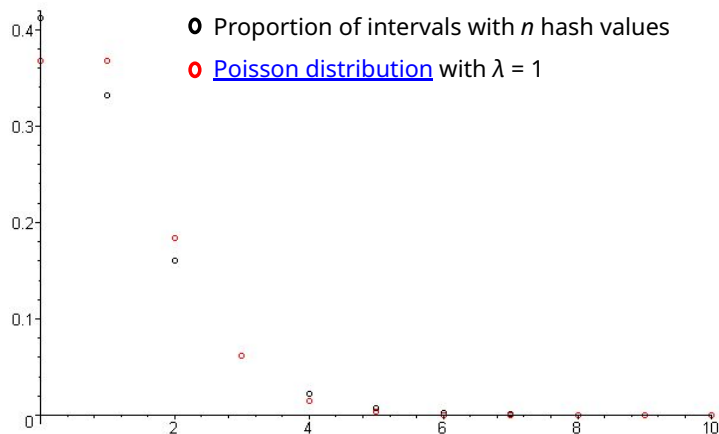
```
unsigned int hash( string const &str ) {  
    unsigned int hash_value = 0;  
    for ( int k = 0; k < str.length(); ++k ) {  
        hash_value = 12347*hash_value + str[k];  
    }  
    return hash_value;  
}
```


String Class

- Is this hash function actually better?
- Suppose we pick n random integers 1 to L
 - ◆ One would expect each integer to appear $\lambda = n/L$ times
 - ◆ Some, however, will appear more often, others less often
- To test whether the integers are random, we will ask:
 - ◆ “How many (what proportion of) integers were chosen k times?”

String Class

- Consider the hash of each of the 354985 string in [single.txt](#) to be a random value in $0, 1, 2, 3, \dots, 2^{32} - 1$
- ◆ Subdivide the integers into groups of approximately 12099
 - ◆ We expect one hash value per interval
 - ◆ Count number of these subintervals which contain 0, 1, 2, ... of these hash values
 - ◆ Plotting these proportions and $1/e \cdot n!$, we see they're very similar



String Class

Problem: Horner's rule runs in $\Theta(n)$

```
"A Elbereth Gilthoniel,\n  Silivren penna miriel\n  O menal aglar elenath!\n  Na-chaered palan-diriel\n  O galadhrimmin ennorath,\n  Fanuilos, le linnathon\n  nef aear, si nef aearon!"
```

Any suggestions to improve the runtime?

J.R.R.
Tolkien

String Class

→ Solution: use characters in locations $2^k - 1$ for $k = 0, 1, 2, \dots$:

```
"A_Elbereth Gilt_honiel,\n  Silivren_penna miriel\n  O menal aglar elenath!\n  Na-chaered palan-diriel\n  O galadhremmin ennorath,\n  Fanuilos, le linnathon\n  nef aear, si nef aearon!"
```

J.R.R.
Tolkien

String Class

→ The runtime is now $\Theta(\log(n))$

```
unsigned int hash( const string &str ) {  
    unsigned int hash_value = 0;  
  
    for ( int k = 1; k <= str.length(); k *= 2 ) {  
        hash_value = 12347*hash_value + str[k - 1];  
    }  
  
    return hash_value;  
}
```

Note: this cannot be used if you require a [cryptographic hash function](#) or [message digest](#)

Arithmetic Hash Function

Unique Identifiers

→ In general, any member variables that are used to uniquely define an object may be used as coefficients in such a polynomial

```
class Person {  
    string surname;  
    string *given_names;  
    unsigned char num_given_names;  
    unsigned short birth_year;  
    unsigned char birth_month;  
    unsigned char birth_day;  
    unsigned int salary;  
    // ...  
};
```

Summary

Hashing Introduction

- Discuss storing unordered data
- Discussed collisions
- Hash functions:
 - ◆ We have seen how number of objects can be mapped to a 32-bit integer
 - ◆ Predetermined hash functions
 - Auto-incremented variables
 - Addresses
 - ◆ Arithmetic hash functions
- Next:
 - ◆ Map 32-bit integer onto a small range
 - ◆ Techniques to handle collisions

Acknowledgement: These slides are adapted from Professor Douglas Wilhelm Harder's ECE250 course at University of Waterloo.